

ІІТМО

Введение в GameDev: ОСНОВЫ ИГРОВОГО ИИ

Лекция 4. Принятие решений и
элементы машинного обучения



1. Теория принятия решений
2. Теория полезности
3. ИИ на основе полезности
4. Машинное обучение
5. Обучение с подкреплением
6. Q-learning

ГЛАВА 4 _ Принятие решений

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений — обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений — обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.



ГЛАВА 4 _ Принятие решений

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений – обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.

Модели принятия
решений



Модели принятия
решений в условиях
определенности

Модели принятия
решений в
условиях риска

Модели принятия
решений в условиях
неопределенности

Определенность ~ полная информация.
Вспоминаем **лекцию 1**.

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений — обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.

Модели принятия
решений

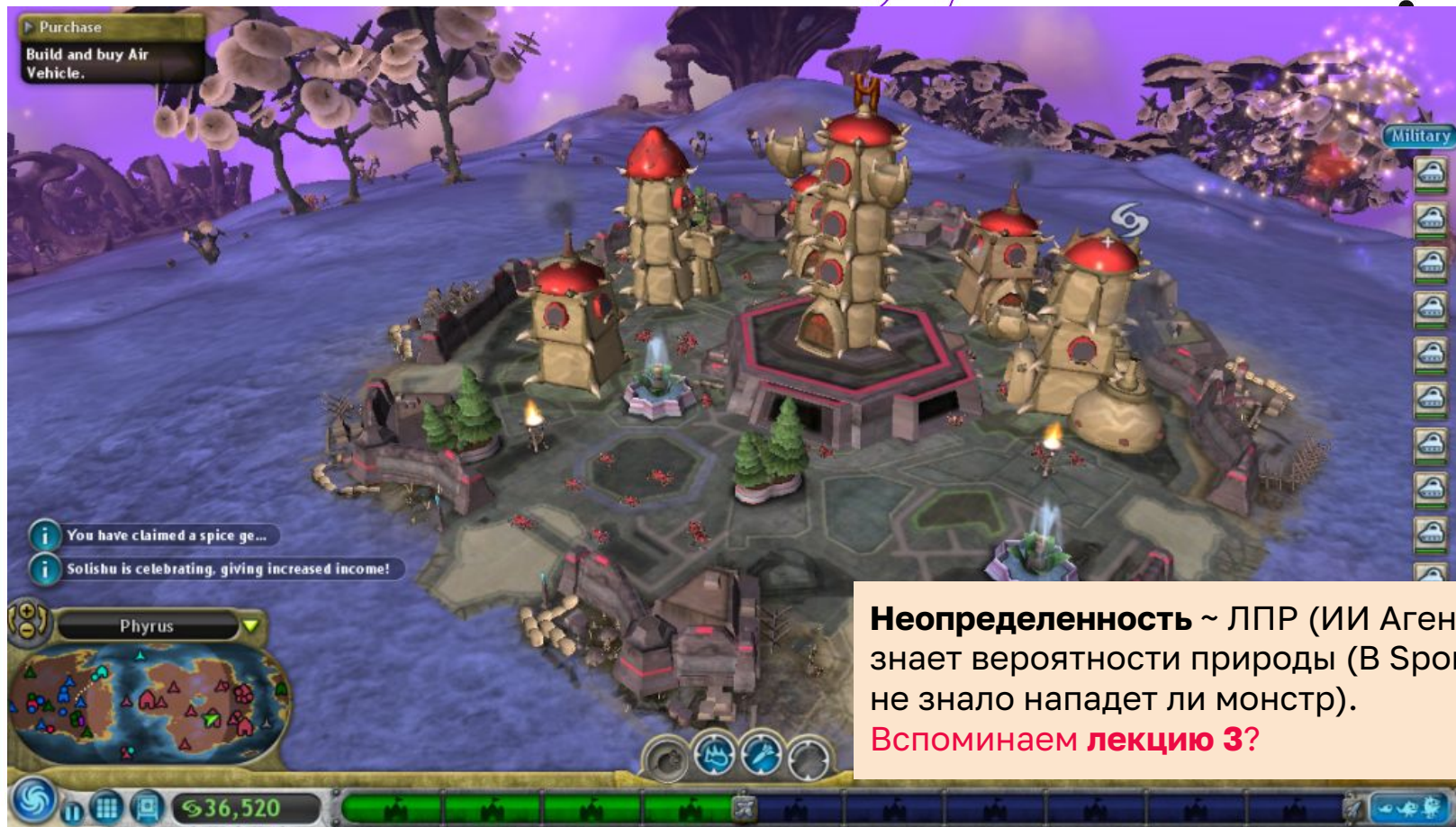
Модели принятия
решений в условиях
определенности

Модели принятия
решений в
условиях риска

Модели принятия
решений в условиях
неопределенности

Определенность ~ полная информация.
Вспоминаем **лекцию 1**.

Неопределенность ~ ЛПР (ИИ Агент) не знает вероятности природы (В Spore ЛПР не знало нападет ли монстр).
Вспоминаем лекцию 3?



МО

собой
дение
бора.

бя

ция.

Неопределенность ~ ЛПР (ИИ Агент) не знает вероятности природы (В Spore ЛПР не знало нападет ли монстр).

Вспоминаем **лекцию 3?**

ГЛАВА 4 _ Принятие решений

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений — обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.



Определенность ~ полная информация.
Вспоминаем **лекцию 1**.

Неопределенность ~ ЛПР (ИИ Агент) не знает вероятности природы (В Spore ЛПР не знало нападет ли монстр).
Вспоминаем лекцию 3?

Риск ~ ЛПР знает вероятности природы

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений — обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.

Риск ~ ЛПР знает вероятности природы

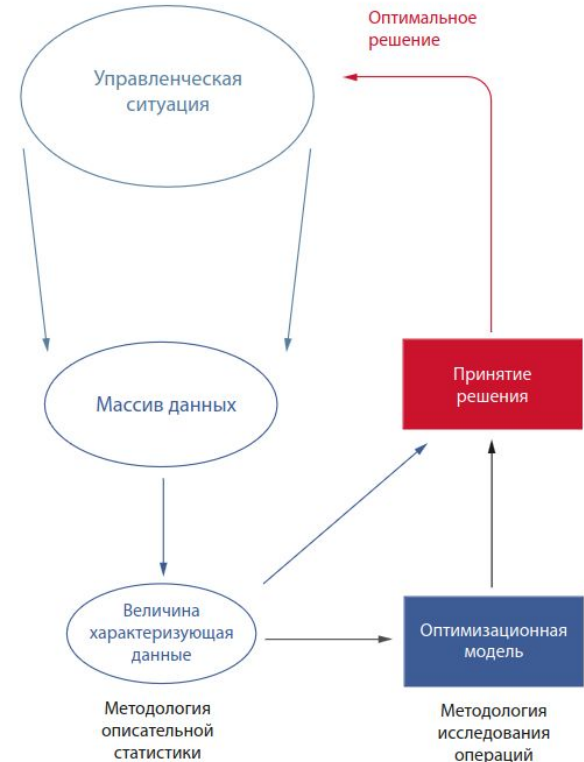
В условиях риска. Известно распределение вероятностей всех состояний природы и для выбора оптимального решения применяют критерии:

- Максимизация ожидаемого результата, выраженного в виде платежей.
- Минимизация ожидаемых потерь.
- Максимизация ожидаемого результата, выраженного в виде значения полезности.

ГЛАВА 4 _ Принятие решений

Теория принятия оптимальных решений* в наиболее общем смысле представляет собой совокупность математических и численных методов, ориентированных на нахождение наилучших вариантов из множества альтернатив и позволяющих избежать их полного перебора.

Теория принятия решений – обширное научное направление, которое включает в себя существенную часть методологии исследования операций и системного анализа.



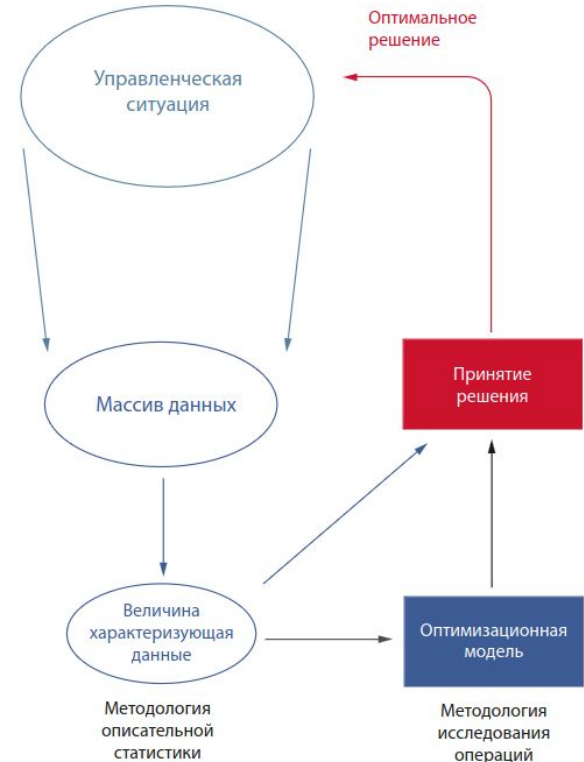
ГЛАВА 4 _ Принятие решений

(Математическая)
Теория оптимизации



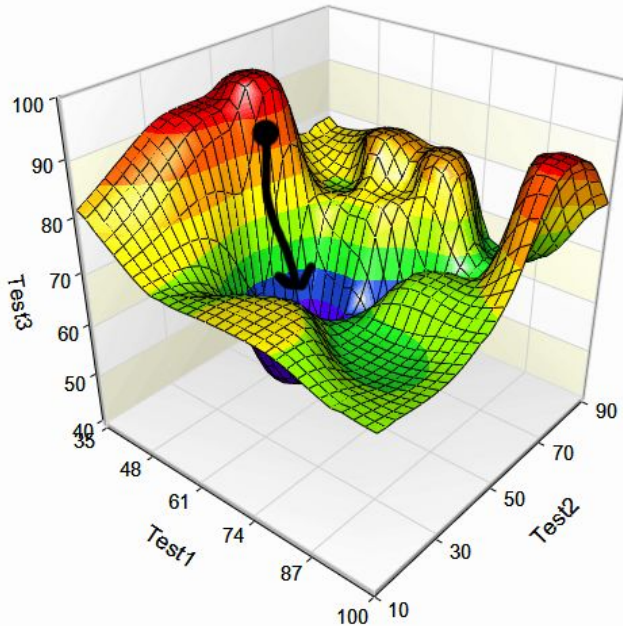
Исследование операций

Теория принятия решений

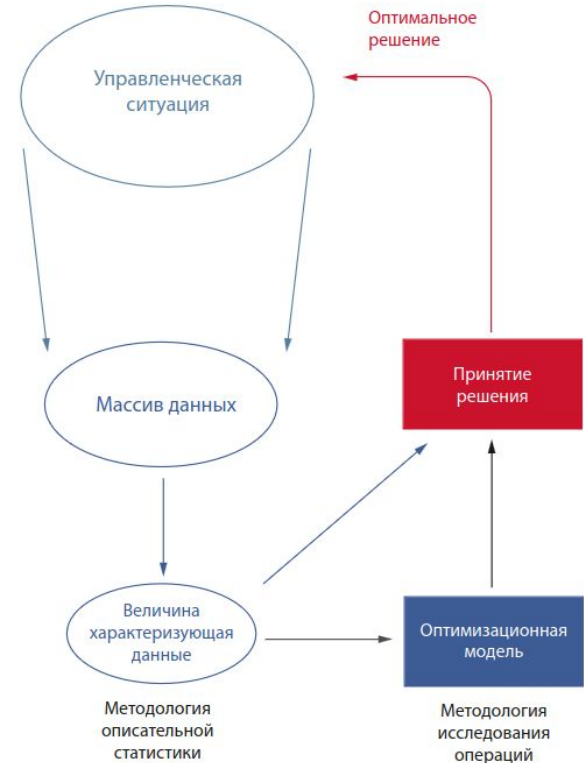


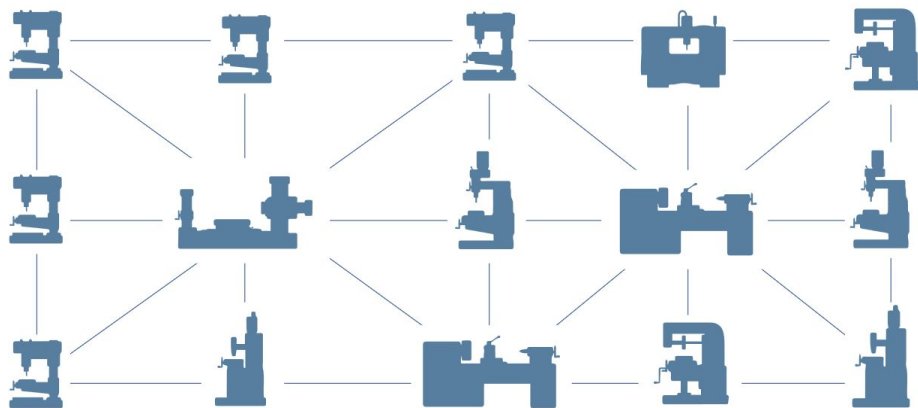
ГЛАВА 4 _ Принятие решений

(Математическая) Теория оптимизации

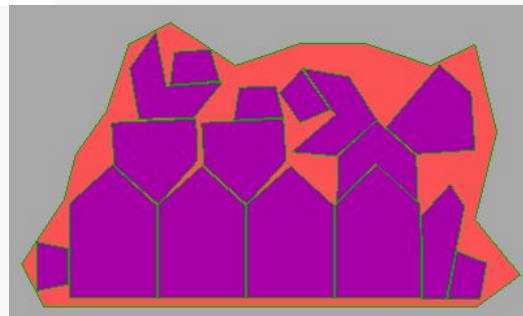
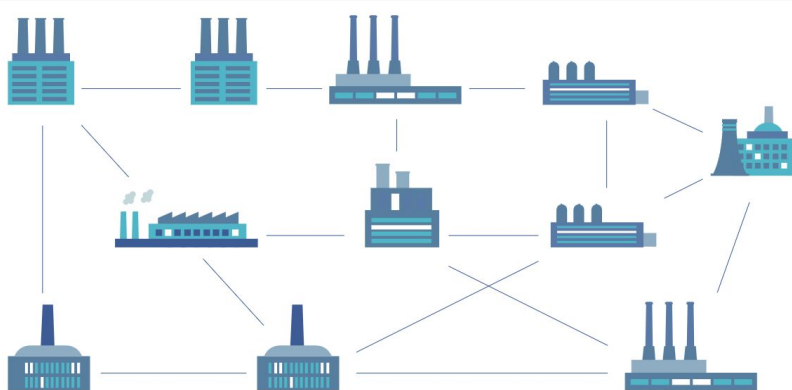


ание операций





Исследование операций

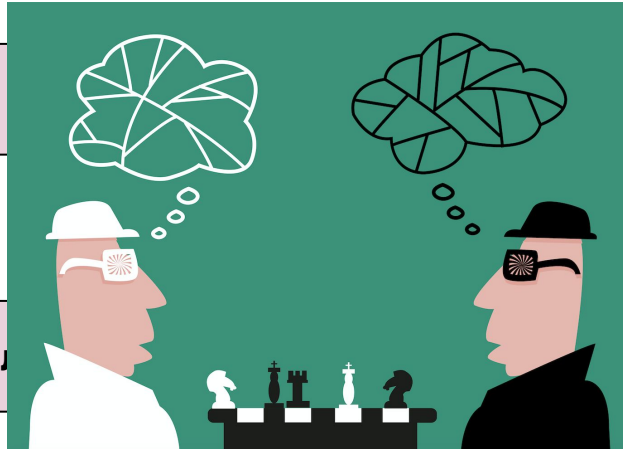


ГЛАВА 4 _ Принятие решений

(Математическая)
Теория оптимизации



Теория принятия решений



ГЛАВА 4 _ Принятие решений

Типовые задачи исследования операций

Модели
распределения

Модели
создания и
хранения запасов

Модели
массового
обслуживания

Модели
замены оборудования
и запчастей

Состязательные
(игровые)
модели

Транспортные
модели

Модели
математического
программирования

Динамические
модели

Теория игр и
теория принятия
решений

Эвристические
методы

Сетевые
модели

Теория массового
обслуживания

Теория вероятностей
и математическая
статистика

Имитационное
моделирование

Теория полезности рассматривает вопросы построения процедур для определения предпочтений в количественной форме.



Полезность, *в нашем случае*, это количественная оценка, когда мы в виде числа выражаем предпочтение ЛПР, пытаюсь отразить его сравнительную природу.

Полезность может измерять что угодно - от наблюдаемых объективных данных (например, здоровье противника) до субъективных понятий, таких как эмоции, настроение и угроза. Различные полезные сведения о возможных действиях или решениях могут быть объединены в линейные или нелинейные формулы и направлять агента к принятию решений на основе агрегированной полезной информации. (AI & Games book)

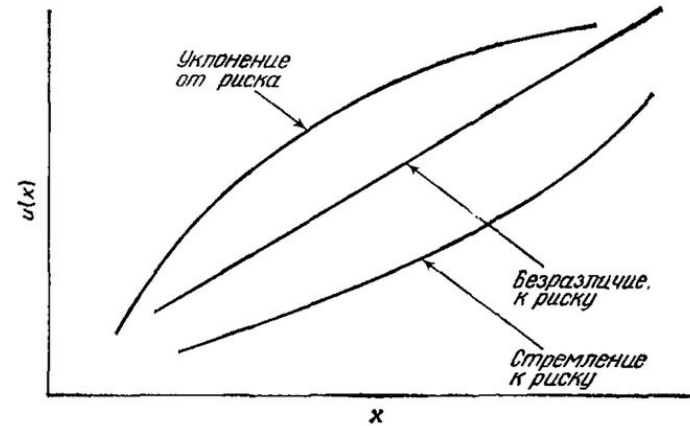
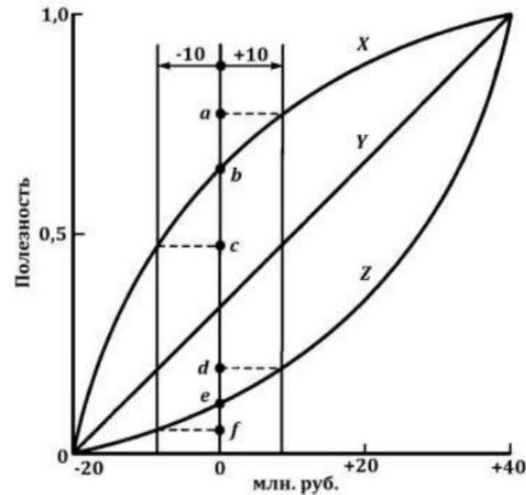
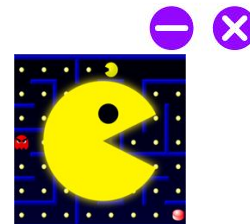
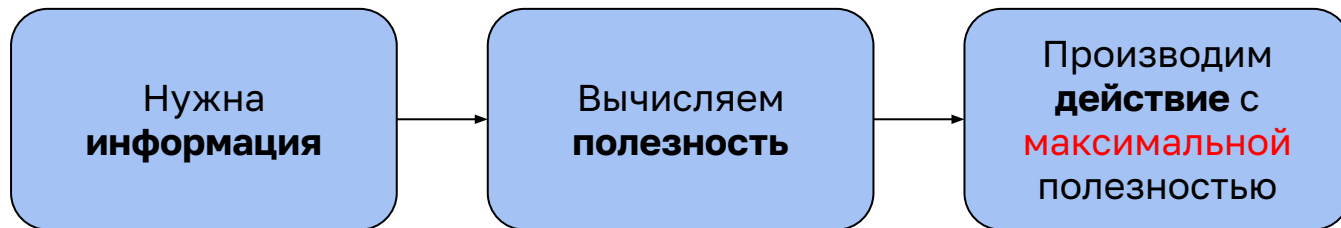
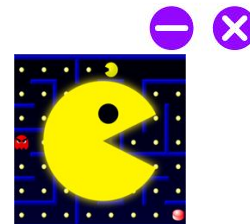
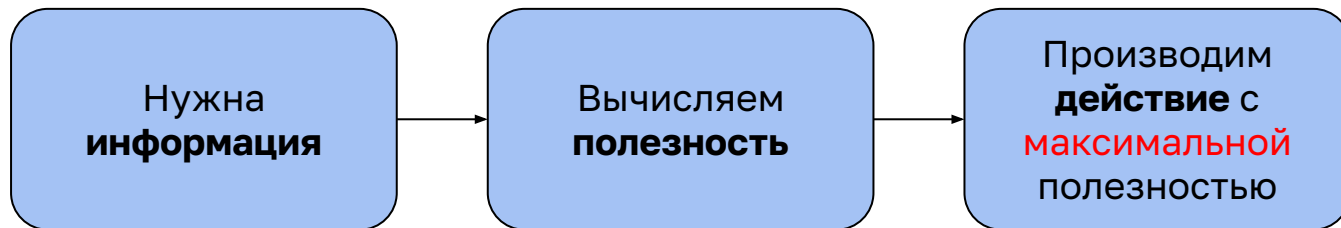


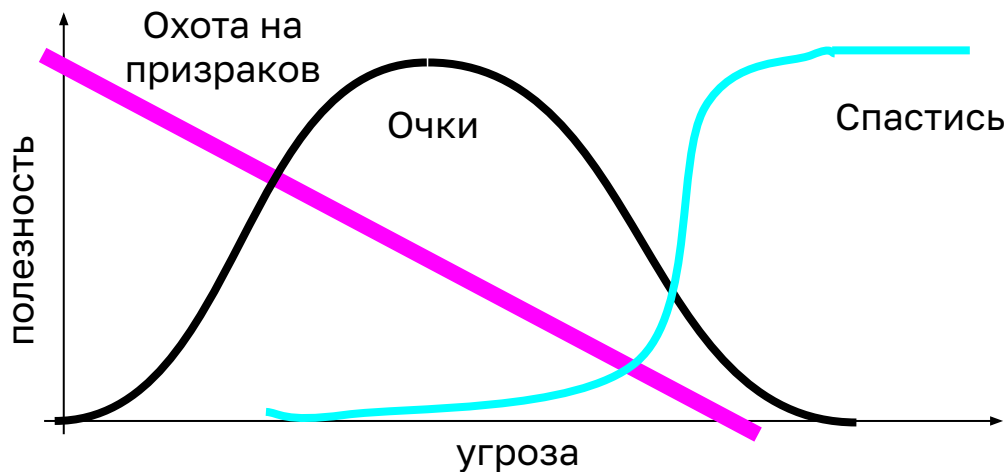
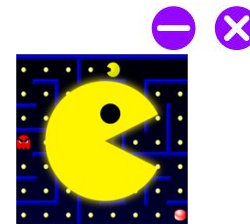
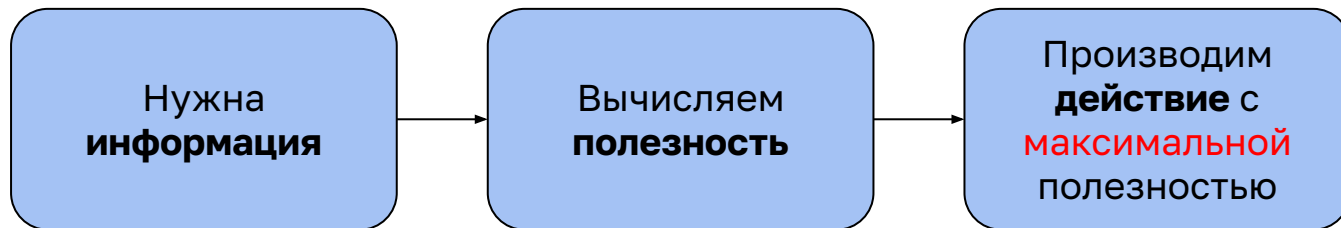
Рис. 4. Отношение к риску.

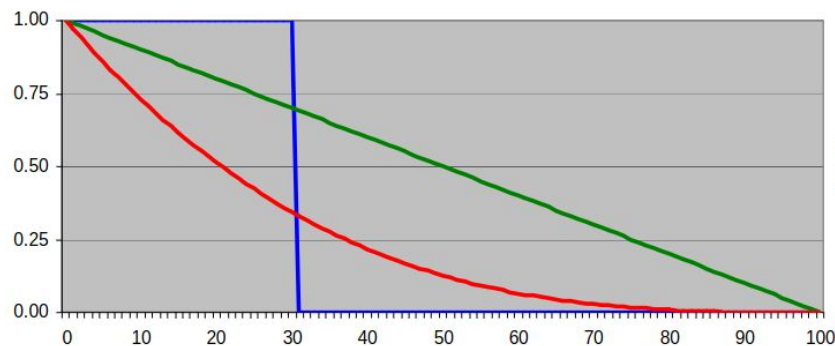
- 1) предварительный анализ для фактической оценки;
- 2) определение соответствующих качественных параметров;
- 3) формирование количественных ограничений;
- 4) выбор функции полезности;
- 5) проверка на согласованность.



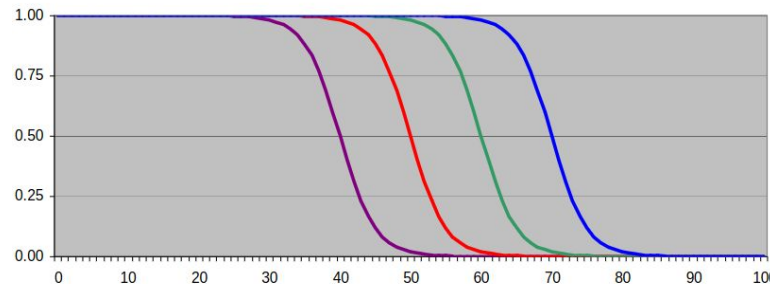
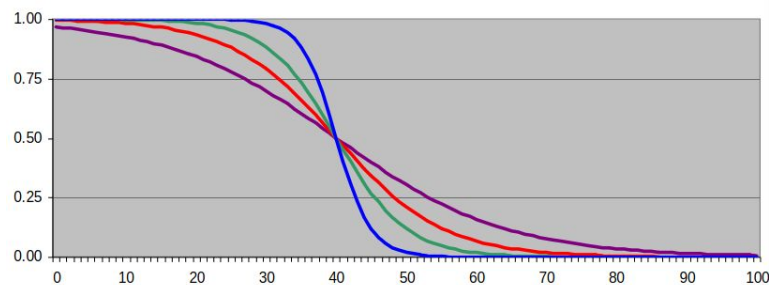


Охота на призраков	Если режим таблетки, то полезность = 100, иначе 0
Сбор очков	Если призрак “далеко”, то полезность = 75, иначе 25
Убежать	Если призрак “близко”, то полезность = 100, иначе 10



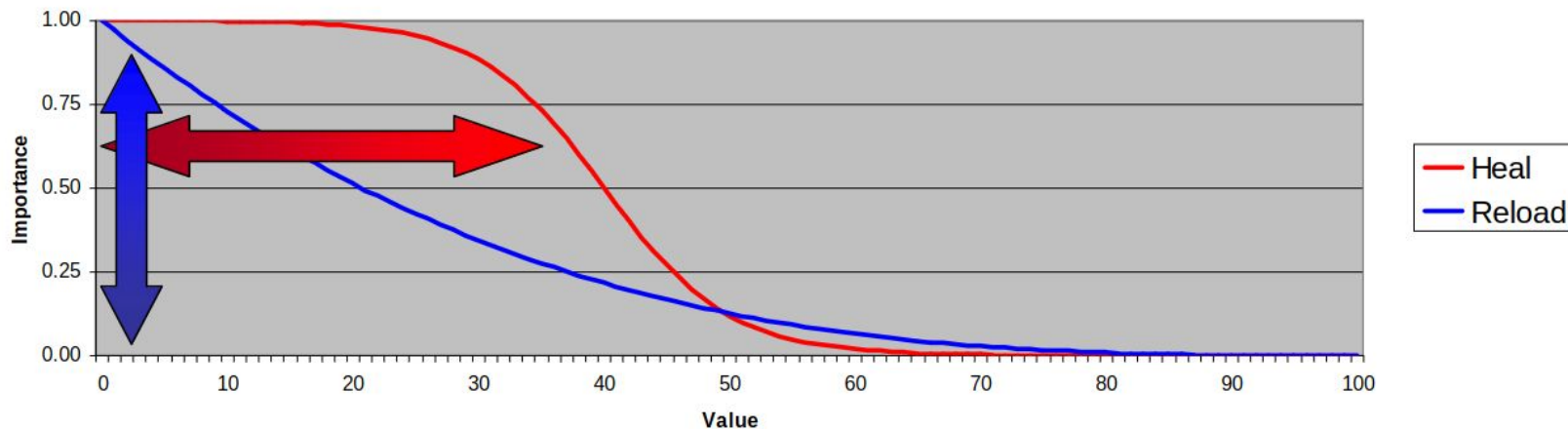


Logistic Function Variations



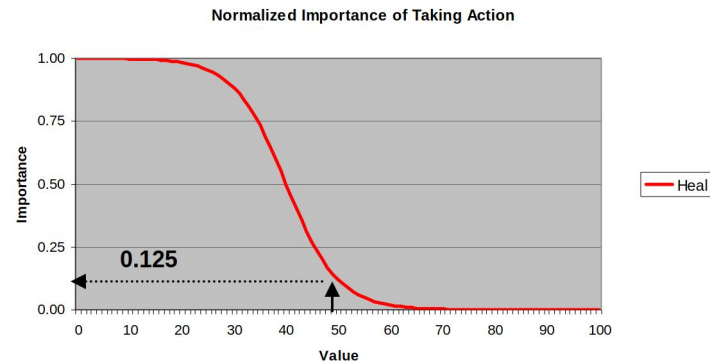
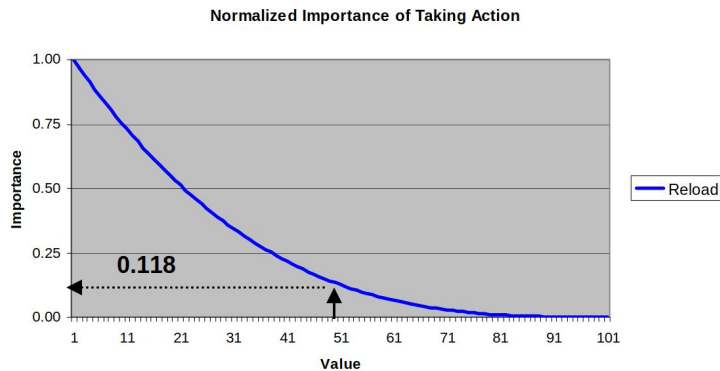
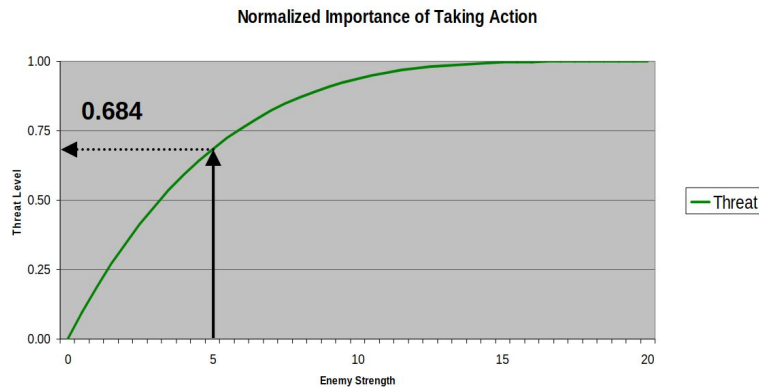


Normalized Importance of Taking Action



ГЛАВА 4 _ Utility AI

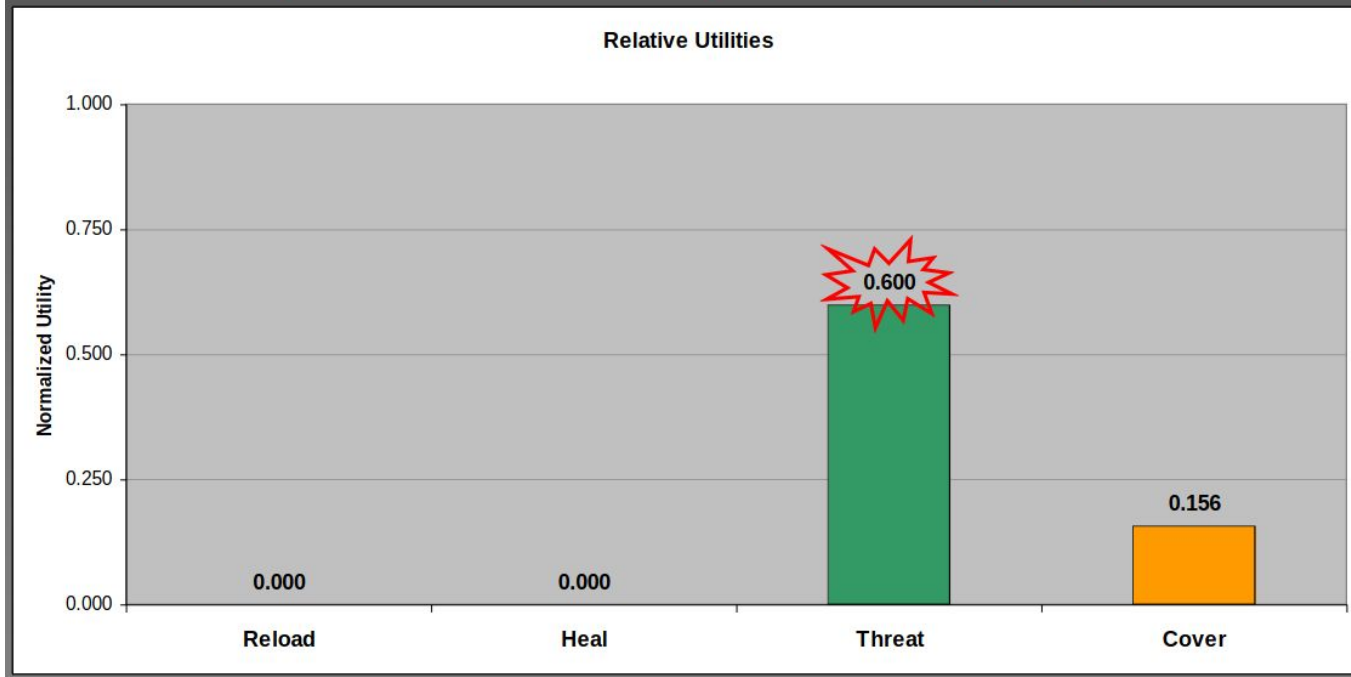
	Enemies	Ammo	Health
Value	5	50	50
Utility	0.684	0.118	0.125



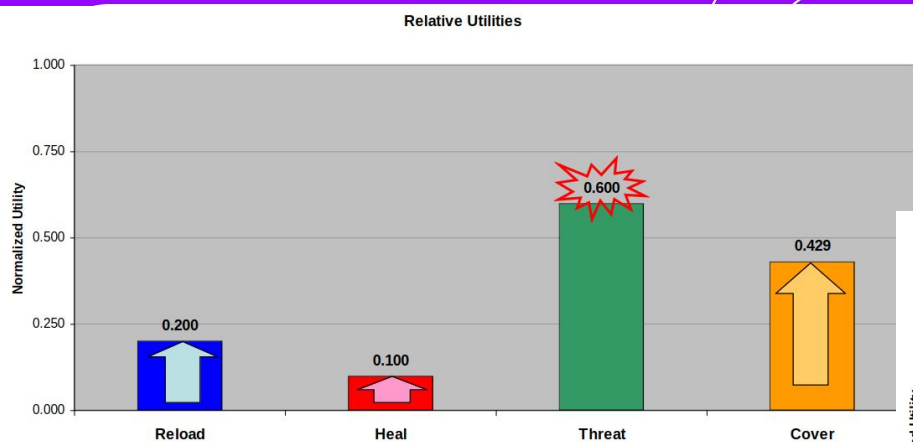


Вопрос: Всегда ли мы можем принять решение на основе только одного фактора? Как объединить факторы?

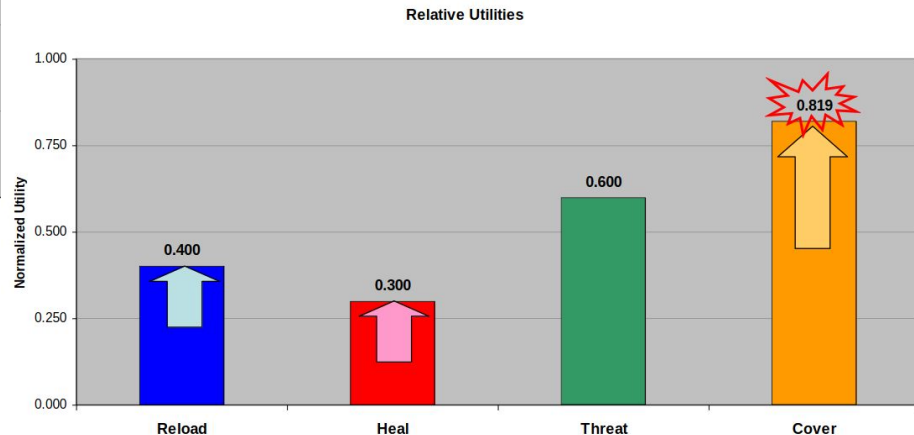
$$\text{Cover} = (0.2 + \text{Reload} + (\text{Heal} \times 1.5)) \times (\text{Threat} \times 1.3)$$



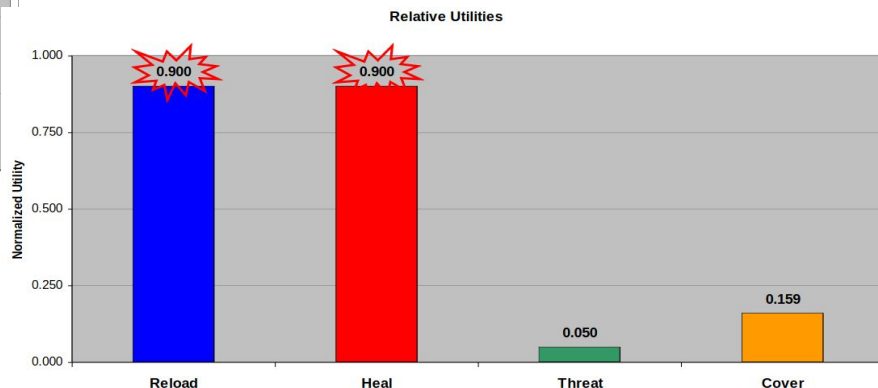
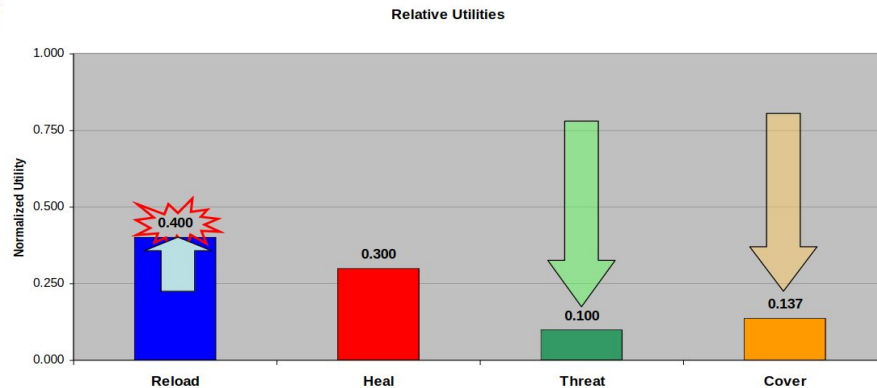
ГЛАВА 4 _ Utility AI



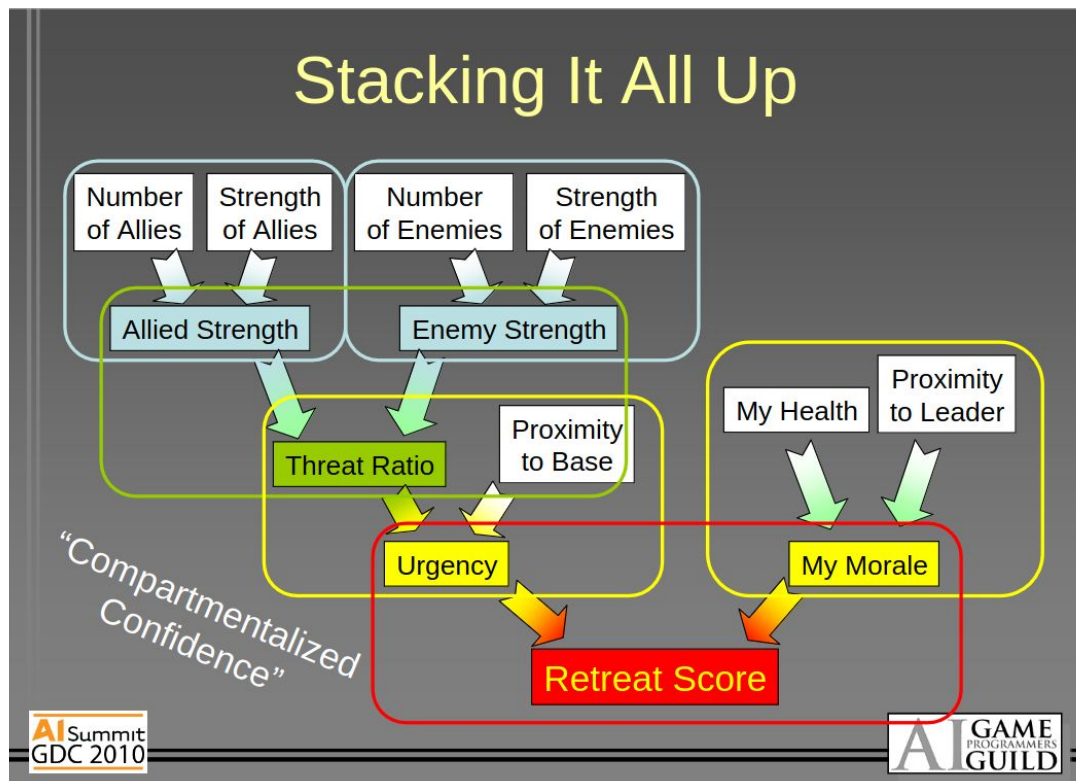
$$(0.2 + 0.2 + 0.15) * (0.78) = 0.429$$



$$\text{Cover} = (0.2 + \text{Reload} + (\text{Heal} \times 1.5)) \times (\text{Threat} \times 1.3)$$



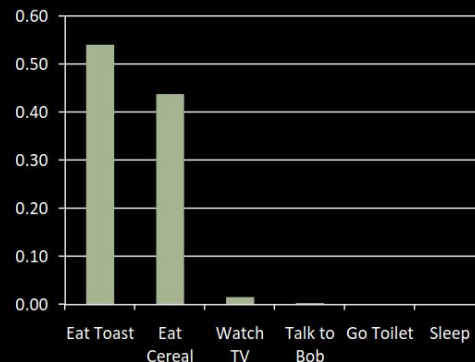
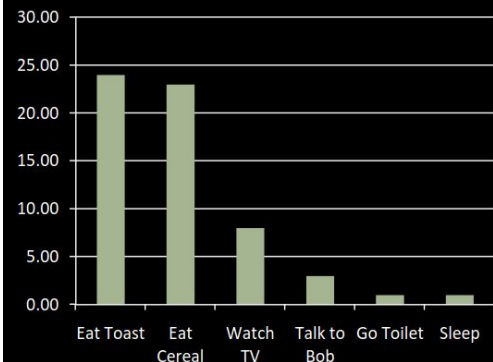
$$\text{Cover} = (0.2 + \text{Reload} + (\text{Heal} \times 1.5)) \times (\text{Threat} \times 1.3)$$





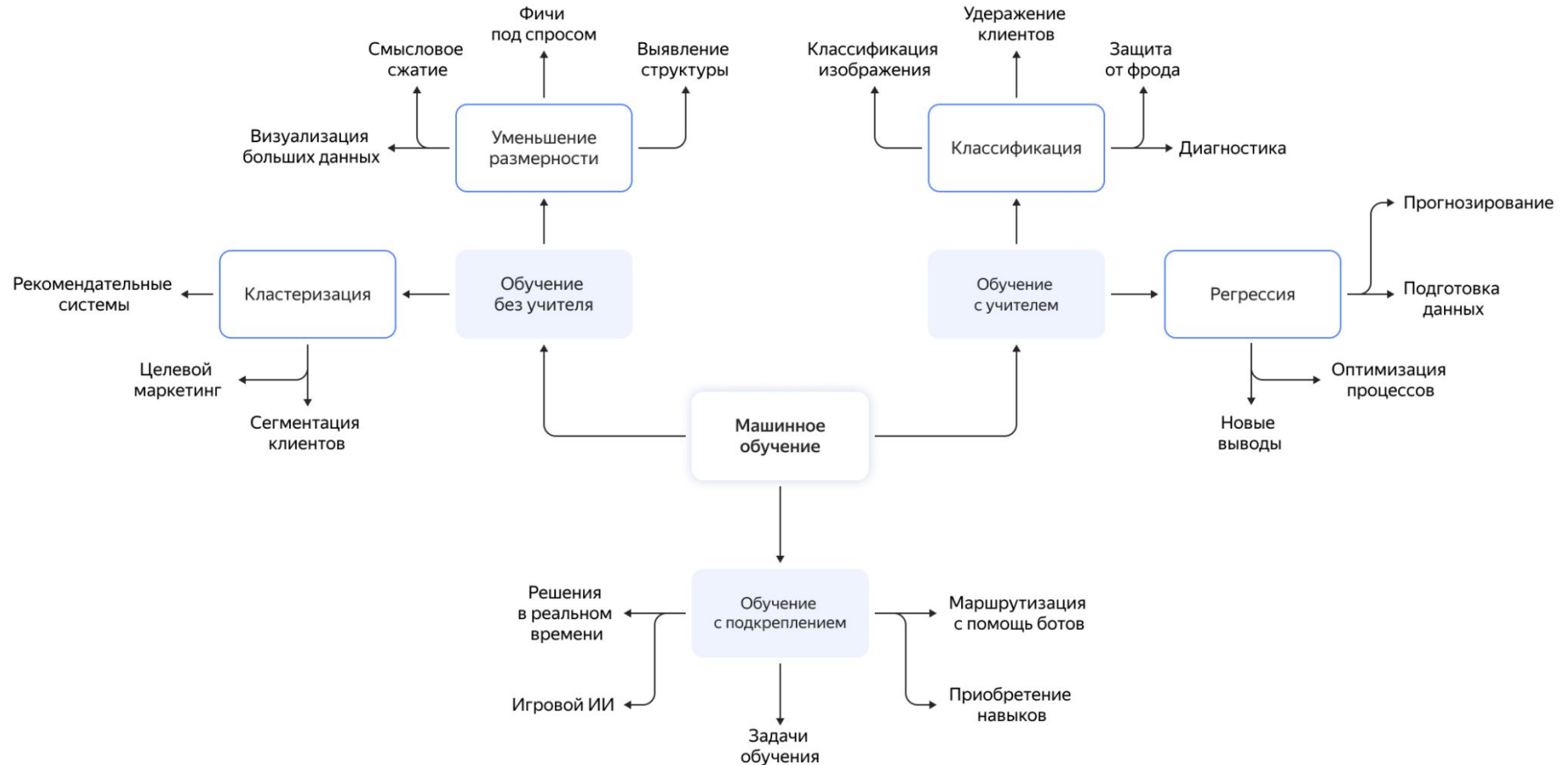
Вопрос: А всегда ли хорошо выполнять действия по максимуму полезности?

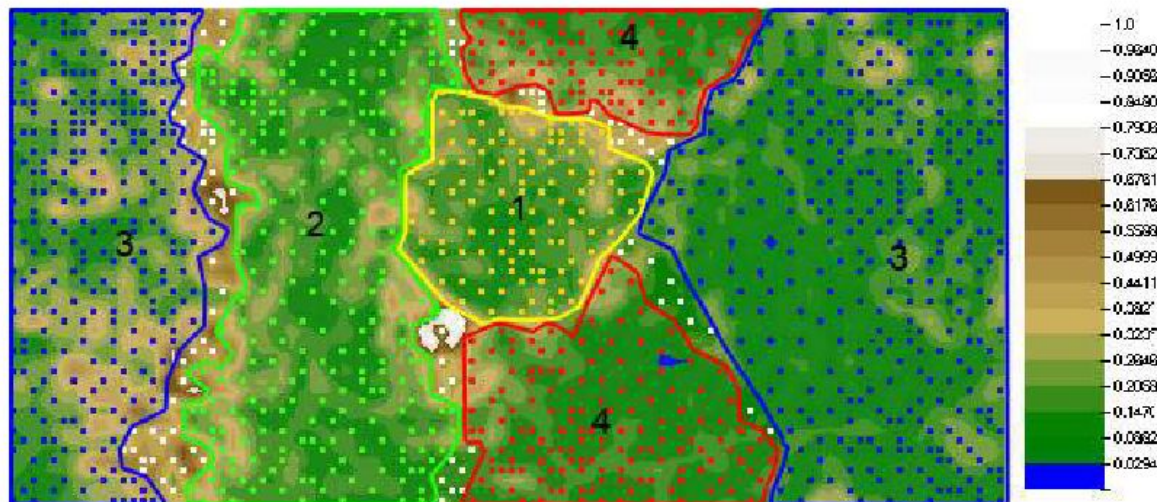
- ❖ Always choose the highest-scoring action
- ❖ Choose randomly from one of the n highest-scoring actions
- ❖ Choose randomly using the score distribution as the probability distribution



Организация ИИ в Sims 3

ГЛАВА 4 _ Машинное обучение





(b) U-matrix visualization of a self-organizing map depicting the four player clusters identified in a population of 1,365 TRU players (shown as small colored squares). Different square colors depict different player clusters. Valleys represent clusters whereas mountains represent cluster borders. Image adopted from [176].

Пример кластеризации
(AI and Games, 2018, p.241)

<https://huggingface.co/learn/ml-games-course/unit0/introduction>

ГЛАВА 4 _ Обучение с подкреплением

Основные источники

Deep RL Course ▾

🔍 Search documentation Ctrl+K

EN ▾ 🌟 3,248

- UNIT 0. WELCOME TO THE COURSE
- UNIT 1. INTRODUCTION TO DEEP REINFORCEMENT LEARNING
 - BONUS UNIT 1. INTRODUCTION TO DEEP REINFORCEMENT LEARNING WITH HUGGY
 - LIVE 1. HOW THE COURSE WORK, Q&A, AND PLAYING WITH HUGGY
- UNIT 2. INTRODUCTION TO Q-LEARNING
- UNIT 3. DEEP Q-LEARNING WITH ATARI GAMES
 - BONUS UNIT 2. AUTOMATIC HYPERPARAMETER TUNING WITH OPTUNA
- UNIT 4. POLICY GRADIENT WITH PYTORCH
- UNIT 5. INTRODUCTION TO UNITY ML-AGENTS
- UNIT 6. ACTOR CRITIC METHODS WITH ROBOTICS ENVIRONMENTS
- UNIT 7. INTRODUCTION TO MULTI-AGENTS AND AI VS AI
- UNIT 8. PART 1 PROXIMAL POLICY OPTIMIZATION (PPO)
- UNIT 8. PART 2 PROXIMAL POLICY OPTIMIZATION (PPO) WITH DOOM
 - BONUS UNIT 3. ADVANCED TOPICS IN REINFORCEMENT LEARNING
- CERTIFICATION AND CONGRATULATIONS

Welcome to the 😊 Deep Reinforcement Learning Course



Welcome to the most fascinating topic in Artificial Intelligence: **Deep Reinforcement Learning**.

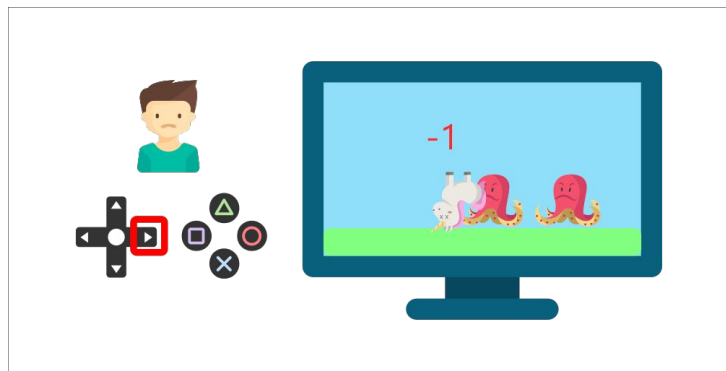
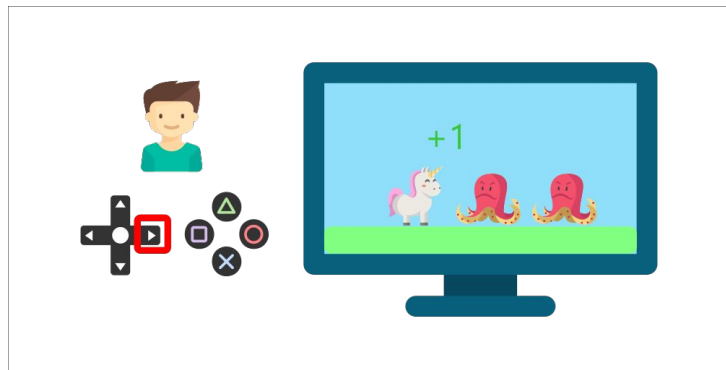
This course will **teach you about Deep Reinforcement Learning from beginner to expert**. It's completely free and open-source!

In this introduction unit you'll:

[Deep RL Course \(huggingface\)](#)

ЦИФРОВАЯ
КУЛЬТУРА
УНИВЕРСИТЕТ ИТМО

Обучение с подкреплением

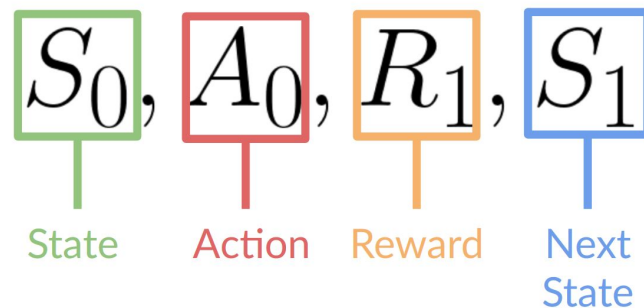
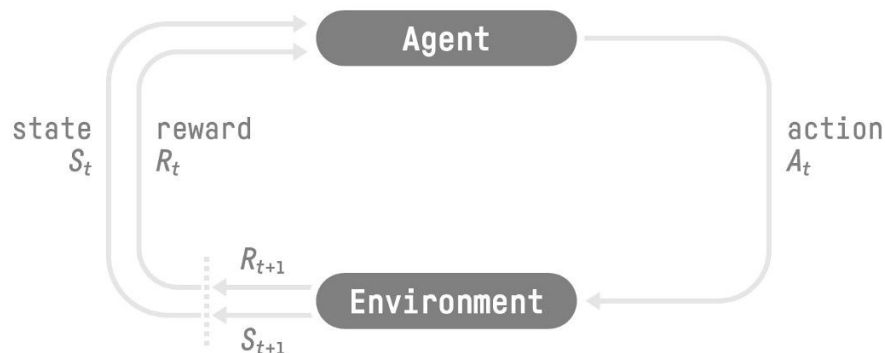


Обучение с подкреплением это всего лишь вычислительный подход к обучению через взаимодействие со средой (environment).

Обучение с подкреплением это инструмент для решения задач управления (задач принятия решений) через построение агентов, которые учатся на основе проб и ошибок, взаимодействуя со средой, и получают награды (положительные и отрицательные) как обратную связь.

Reinforcement learning is a framework for solving control tasks (also called decision problems) by building agents that learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.

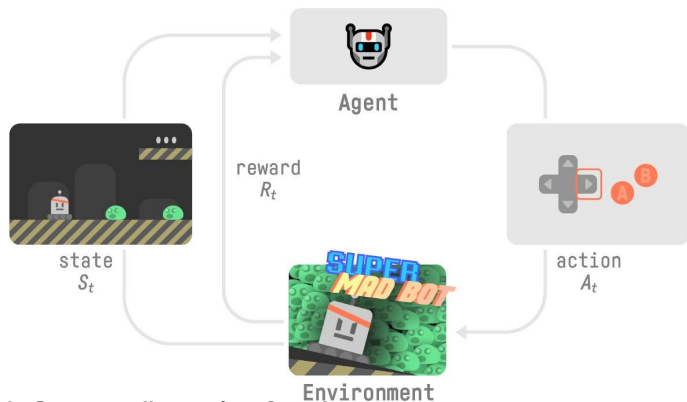
ГЛАВА 4 _ Обучение с подкреплением



Почему целью агента является максимизация ожидаемой прибыли?

Потому что RL основан на **гипотезе вознаграждения**, которая заключается в том, что все цели могут быть описаны как максимизация ожидаемой отдачи (ожидаемое кумулятивное вознаграждение).

Вот почему при обучении с подкреплением, чтобы добиться наилучшего поведения, мы стремимся научиться предпринимать действия, которые максимизируют ожидаемое совокупное вознаграждение.



Observation Space

State: complete description of the state of the world (no hidden information).



Observation: partial description of the state of the world.



Action Space

Discrete: finite number of possible actions



Continuous: infinite number of possible actions



Кумулятивное вознаграждение на каждом временном шаге t может быть записано как:

$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$



Return: cumulative reward

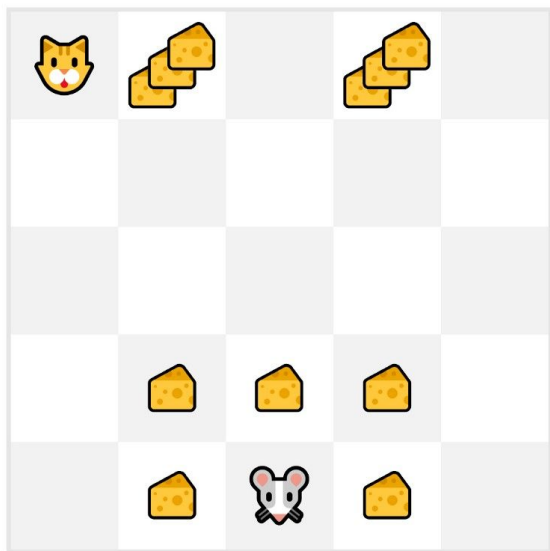
Trajectory (read Tau)

Sequence of states and actions

$$R(\tau) = \sum_{k=0}^{\infty} r_{t+k+1}$$

ГЛАВА 4 _ Обучение с подкреплением

Однако на самом деле мы не можем просто добавить их таким образом. Награды, которые приходят раньше (в начале игры), с большей вероятностью произойдут, поскольку они более предсказуемы, чем долгосрочное будущее вознаграждение.

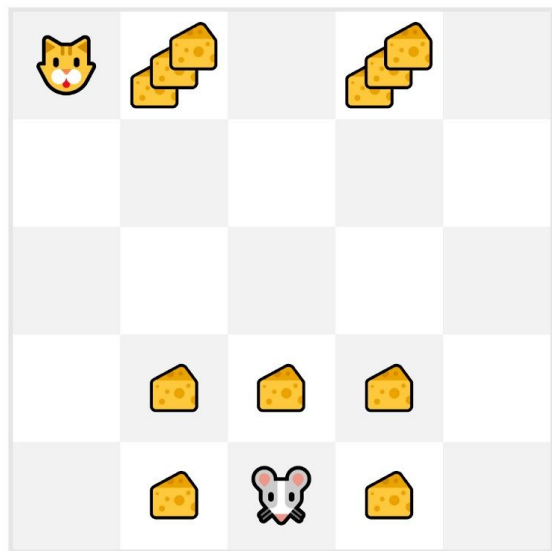


Как мы можем видеть на карте, более вероятно съесть сыр рядом с нами, чем сыр рядом с кошкой (чем ближе мы к кошке, тем это опаснее).

Следовательно, награда рядом с кошкой, даже если она больше (больше сыра), будет более дешевой, поскольку мы на самом деле не уверены, что сможем ее съесть.

ГЛАВА 4 _ Обучение с подкреплением

Однако на самом деле мы не можем просто добавить их таким образом. Награды, которые приходят раньше (в начале игры), с большей вероятностью произойдут, поскольку они более предсказуемы, чем долгосрочное будущее вознаграждение.



Скидка на вознаграждения:

1. Ставка дисконтирования (какова скидка)
0~1 (как правило 0.95~0.99)
2. Каждое последующее дисконтирование по ставке по временному шагу.

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$



Return: cumulative reward

Gamma: discount rate

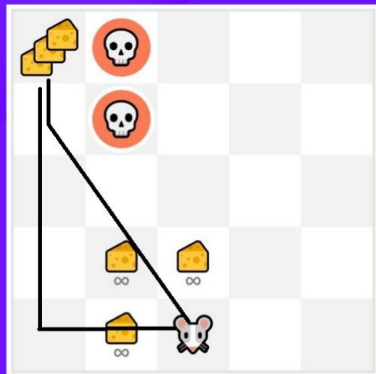
Trajectory (read Tau)

Sequence of states and actions

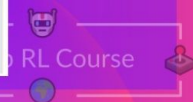
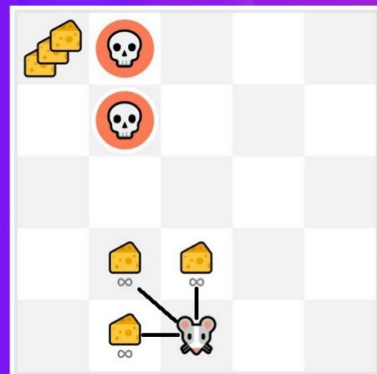
$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

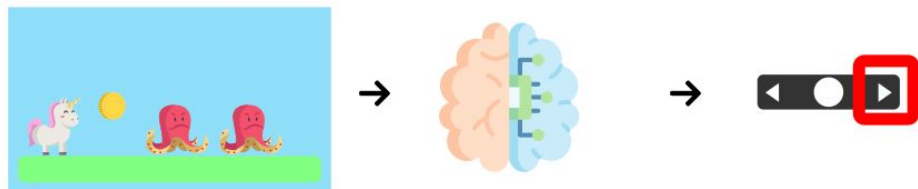
Exploration/ Exploitation tradeoff

Exploration: trying random actions in order to find more information about the environment.



Exploitation: using known information to maximize the reward.





State $\rightarrow \pi(\text{State}) \rightarrow \text{Action}$

Существует два подхода к обучению нашего агента поиску этой оптимальной политики (стратегии) π^* :

1. Непосредственно, обучая агента тому, какие действия следует предпринять, учитывая текущее состояние: **Policy-Based Methods**.
2. Косвенно обучая агента узнавать, какое состояние является более ценным, а затем предпринимать действия, которые приводят к более ценным состояниям: **Value-Based Methods**.

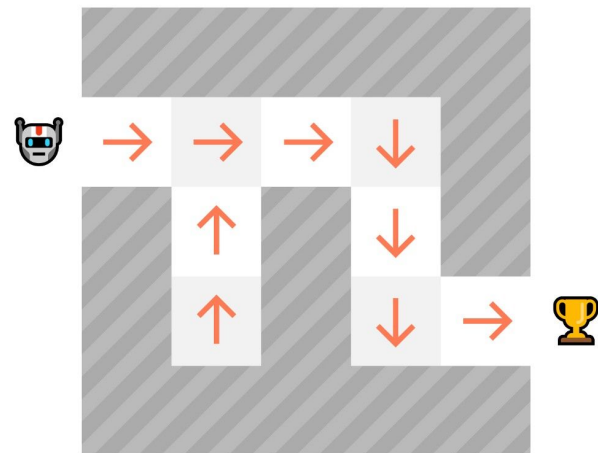
Policy-Based Methods

Deterministic: a policy at a given state will always return the same action.

$$a = \pi(s)$$



State $s_0 \rightarrow \pi(s_0) \rightarrow a_0 = \text{Right}$



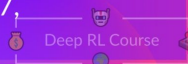
Stochastic: output a probability distribution over actions.

$$\pi(a|s) = P[A=s]$$

Probability Distribution over the set of actions given the state



State $s_0 \rightarrow \pi(A|s_0) \rightarrow [\text{Left: } 0.1, \text{Right: } 0.7, \text{Jump: } 0.2]$



Как решать?

Policy Based

Value-Based Methods

The value of a state is the **expected discounted return** the agent can get if it **starts in that state**, and then act according to our policy.

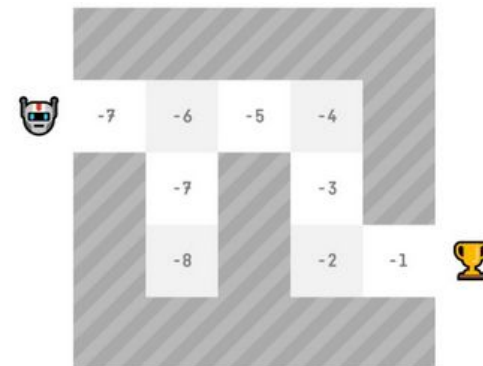
$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

Expected discounted return

Starting
at state s

“Act according to our policy” just means that our policy is “going to the state with the highest value”.

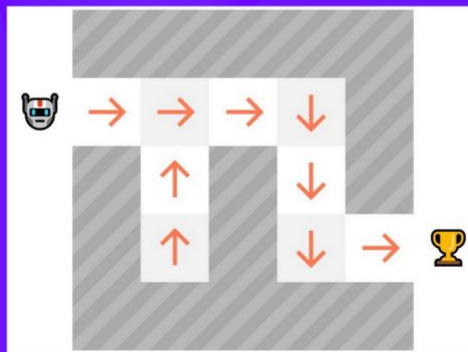


Value Based

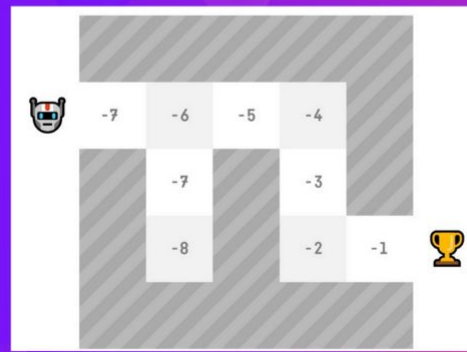
Как решать?

Two approaches to find optimal policy π^* :

Policy-Based methods: train the agent to learn which **action to take**, given a state.



Value-Based methods: train the agent to learn which state is **more valuable** and take the action that leads to it.



Policy-based method:

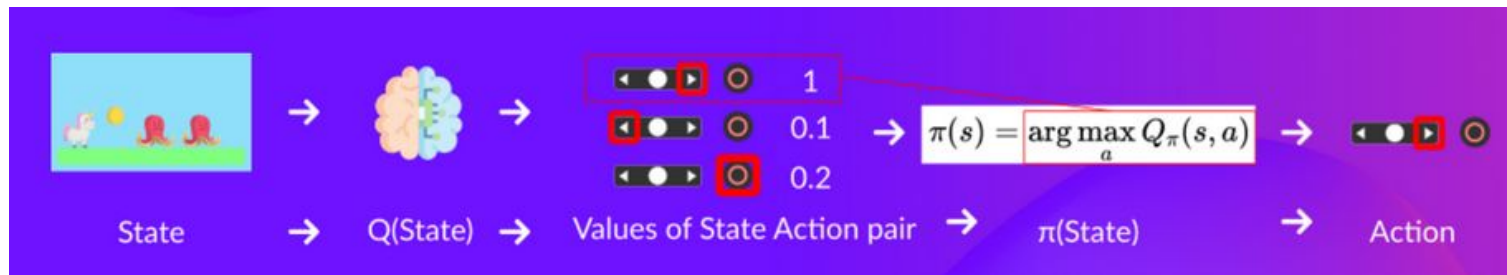
- Обучаем саму “политику”

Связь между методами:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Value-based method:

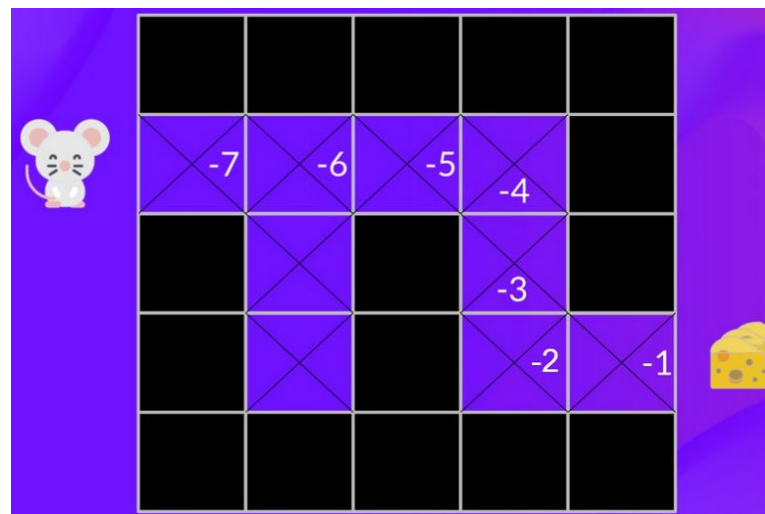
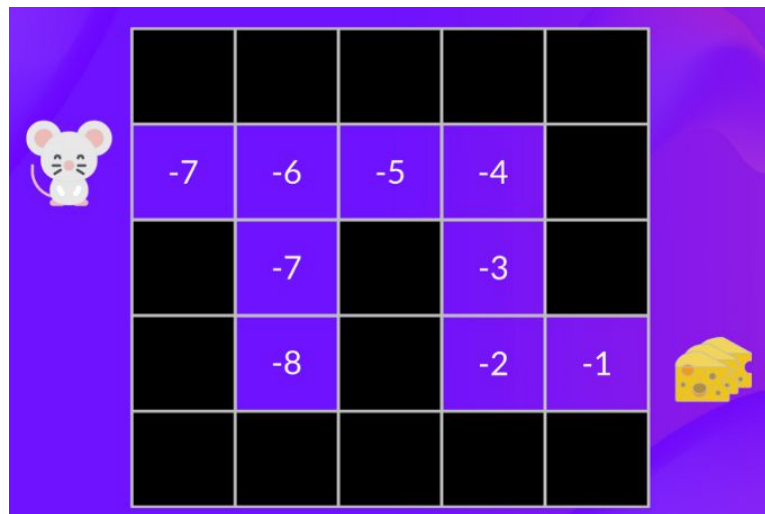
- Обучаем функцию значений



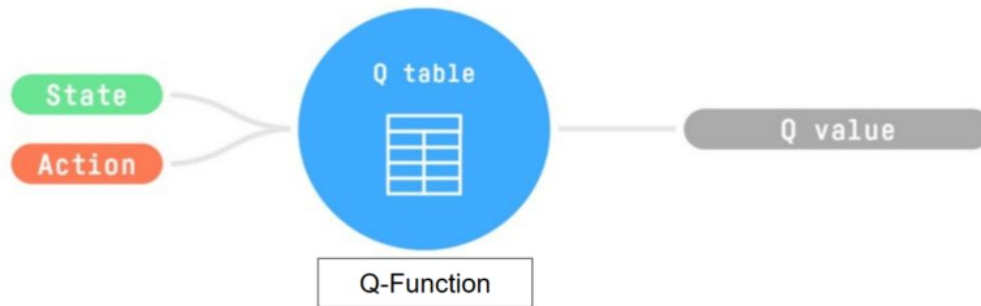
ГЛАВА 4 _ Обучение с подкреплением

Существуют два основных value-based подхода:

Функция состояние-значение: **Функция действие-значение:**



Q-Function



To train this Q-Function, that given a state and action as input, output the value, we use the Q-Learning algorithm.



ГЛАВА 4 _ Q-learning

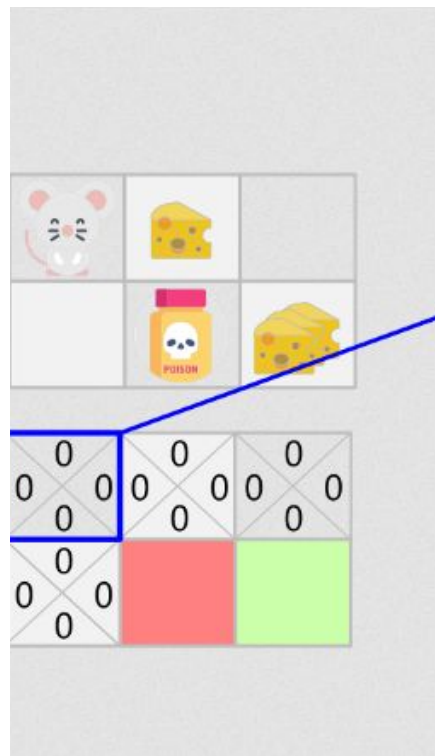
- **Value состояния или пары состояние-действие** - это ожидаемое совокупное вознаграждение, которое получает наш агент, если он запускается в этом состоянии (или паре состояние-действие) и затем действует в соответствии со своей политикой.
- **Вознаграждение** - это обратная связь, которую агент получает от среды после выполнения действия в состоянии.



				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-таблица

ГЛАВА 4 _ Q-learning



States

Actions

				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

ГЛАВА 4 _ Q-learning

В общем, **Q-Learning** - это алгоритм RL, который:

- Обучает Q-функцию (функцию действия-значения), которая внутренне представляет собой Q-таблицу, содержащую все значения пары состояние-действие.
- Учитывая состояние и действие, наша Q-функция выполнит поиск соответствующего значения в своей Q-таблице.
- По завершении обучения у нас будет оптимальная Q-функция, что означает, что у нас есть оптимальная Q-таблица.
- И если у нас есть оптимальная Q-функция, то у нас есть оптимальная политика, поскольку мы знаем, какие действия лучше всего предпринять в каждом состоянии.

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Algorithm 14: Sarsamax (Q-Learning)

Input: policy π , positive integer $num_episodes$, small positive fraction α , GLIE $\{\epsilon_i\}$

Output: value function Q ($\approx q_\pi$ if $num_episodes$ is large enough)

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

for $i \leftarrow 1$ to $num_episodes$ do

$\epsilon \leftarrow \epsilon_i$

 Observe S_0

$t \leftarrow 0$

 repeat

 Choose action A_t using policy derived from Q (e.g., ϵ -greedy) Step 2

 Take action A_t and observe R_{t+1}, S_{t+1} Step 3

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$ Step 4

$t \leftarrow t + 1$

 until S_t is terminal;

end

return Q

↖ Step 1

ГЛАВА 4 _ Q-learning

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

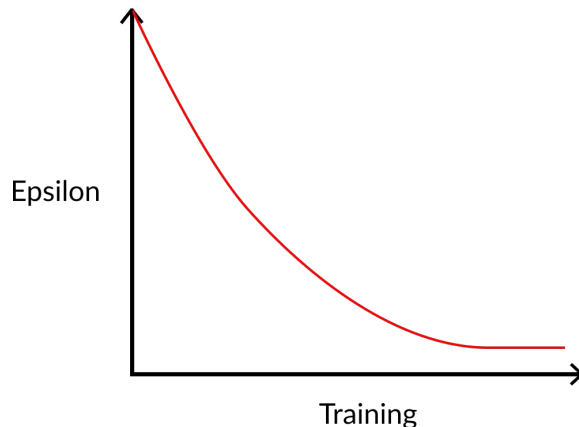
				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

ГЛАВА 4 _ Q-learning

Choose action A_t using policy derived from Q (e.g., ϵ -greedy)

Используя **эпсилон-жадную** политику выбираем следующее действие A :

1. Генерируем случайное число от 0 до 1.
2. С вероятностью **эпсилон** - агент исследует (**exploration**) мир (выбирает случайное действие)
3. С вероятностью (**1 - эпсилон**) - агент использует накопленные знания (**exploitation**), то есть выбирает действие жадно.



Take action A_t and observe R_{t+1}, S_{t+1}

$$\underbrace{Q(S_t, A_t)}_{\text{New Q-value estimation}} \leftarrow \underbrace{Q(S_t, A_t)}_{\text{Former Q-value estimation}} + \underbrace{\alpha}_{\text{Learning Rate}} \underbrace{[R_{t+1} + \gamma \max_a Q(S_{t+1}, a)]}_{\text{Discounted Estimate of next state}} - \underbrace{Q(S_t, A_t)}_{\text{Former Q-value estimation}}$$

Algorithm 14: Sarsamax (Q-Learning)

Input: policy π , positive integer $num_episodes$, small positive fraction α , GLIE $\{\epsilon_i\}$

Output: value function Q ($\approx q_\pi$ if $num_episodes$ is large enough)

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

for $i \leftarrow 1$ to $num_episodes$ do

$\epsilon \leftarrow \epsilon_i$

 Observe S_0

$t \leftarrow 0$

 repeat

 Choose action A_t using policy derived from Q (e.g., ϵ -greedy) Step 2

 Take action A_t and observe R_{t+1}, S_{t+1} Step 3

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$ Step 4

$t \leftarrow t + 1$

 until S_t is terminal;

end

return Q

↖ Step 1

ГЛАВА 4 _ Q-learning



- 1) Всегда стартуем с одной точки
- 2) Цель съесть большой кусок сыра и избегать яда
- 3) $\text{learning_rate} = 0.1$
- 4) $\text{gamma} = 0.99$

				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

+0: на клетку без всего

+ 1: съел маленький кусок сыра

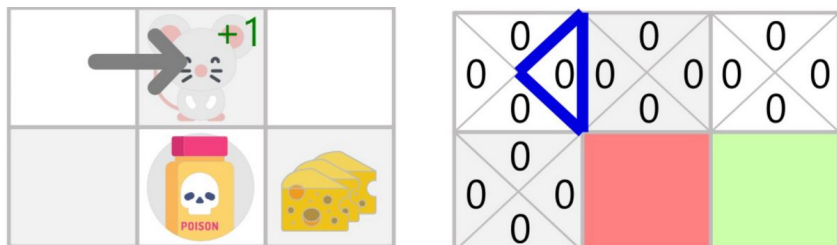
+10: съел большой кусок сыра

-10: съел яд

+0: для любого действия, если уж было совершено 5 действий

ГЛАВА 4 _ Q-learning

Выбираем действие (эпсилон = 1)



$R[t+1] = 1$ - агент съел маленький кусок сыра

Обновляем $Q(S_t, A_t)$

$$Q(Q[0], \text{right}) = 0 + 0.1 \cdot (1 + 0.99 \cdot 0 - 0)$$

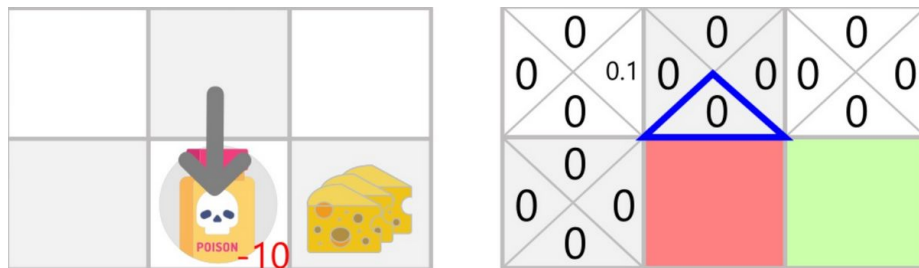
$$Q(Q[0], \text{right}) = 0.1$$

	←	→	↑	↓
	0	0.1	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$$

ГЛАВА 4 _ Q-learning

Выбираем действие (эпсилон = 1)



$R[t+1] = -10$ - агент съел яд (конечный state)

Обновляем $Q(S_t, A_t)$

$$Q(Q[1], \text{down}) = 0 + 0.1 \cdot (-10 + 0.99 \cdot 0 - 0)$$

$$Q(Q[1], \text{down}) = -1$$

				
	0	0.1	0	0
	0	0	0	-1
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$$

Algorithm 14: Sarsamax (Q-Learning)

Input: policy π , positive integer $num_episodes$, small positive fraction α , GLIE $\{\epsilon_i\}$

Output: value function Q ($\approx q_\pi$ if $num_episodes$ is large enough)

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

for $i \leftarrow 1$ to $num_episodes$ do

$\epsilon \leftarrow \epsilon_i$

 Observe S_0

$t \leftarrow 0$

 repeat

 Choose action A_t using policy derived from Q (e.g., ϵ -greedy) Step 2

 Take action A_t and observe R_{t+1}, S_{t+1} Step 3

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$ Step 4

$t \leftarrow t + 1$

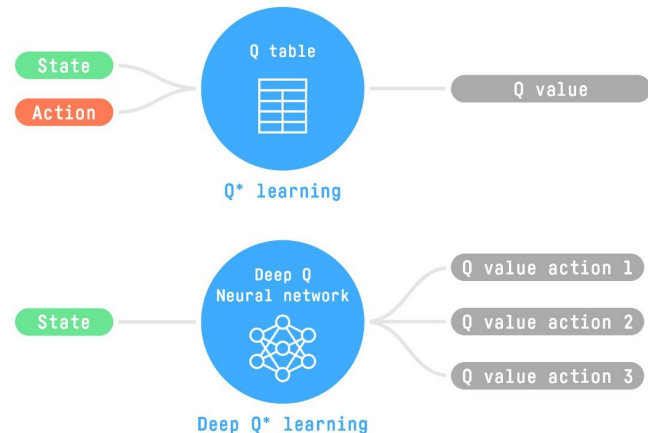
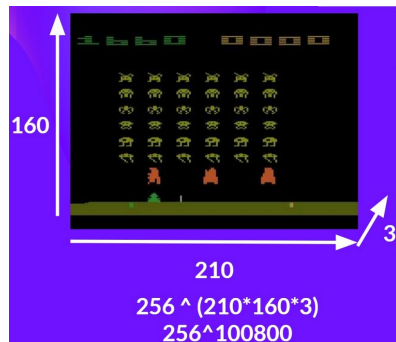
 until S_t is terminal;

end

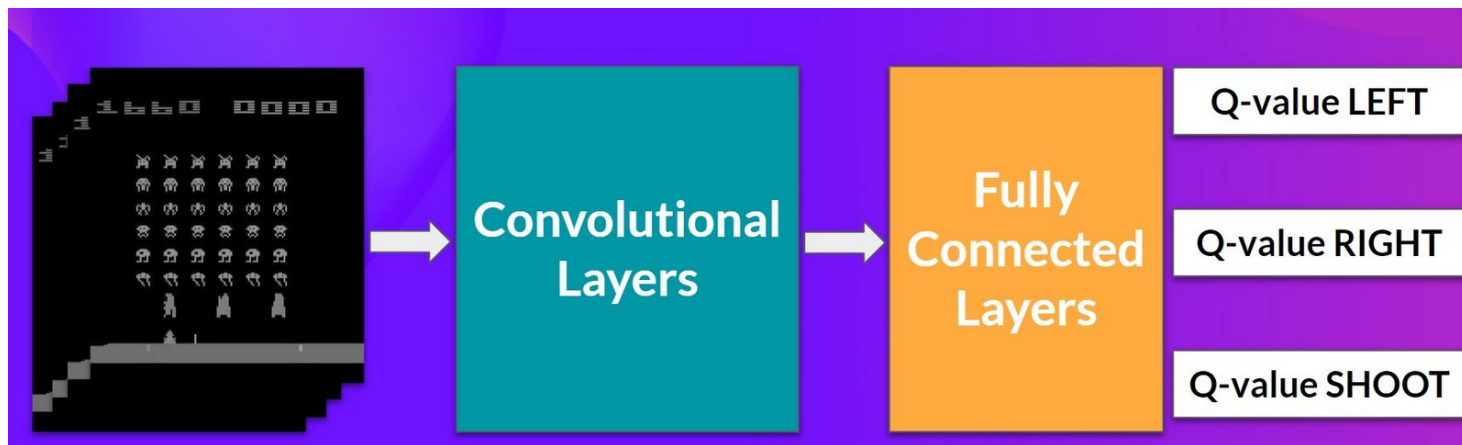
return Q

↖ Step 1

ГЛАВА 4 _Deep Q-learning



<https://cs.stanford.edu/people/karpathy/convnetjs/demo/rldemo.html>



ГЛАВА 4 _ ПР2. Принятие решений

ПР2. Принятие решений (7 баллов)

Вариант 1. Закрепление материала.

Задание 1. ИИ на основе полезности. Спроектируйте ИИ на основе полезности для некоторого NPC:

- Описание NPC, действия, функции полезности (1 балл)
- Есть хотя бы одно композитное действие (1 балл)
- Приведено несколько примеров при различных входных данных (1 балл)

Задание 2. Q-learning. Изучите правила и запустите моделирование на ресурсе <https://www.mladdict.com/q-learning-simulator> на 15-20 эпохах. Сохраните полученную матрицу. Приведите 1-2 итерации (не эпохи!) ручным счетом (как в лекции). (2 балла)
Обучите агента 2-3 раза с нуля на сервисе с разными настраиваемыми гипер параметрами на 10-20 эпохах. Сохраняйте результирующие матрицы. Сделайте вывод о влиянии гиперпараметров. Для каждой результирующей матрицы постройте матрицу “политик” и цепочку состояний и действий на основе этих политик от начального состояния до конечного состояния. (2 балла)

ГЛАВА 4 _ ПР2. Принятие решений

ПР2. Принятие решений (7 баллов)

Вариант 2. Для тпу прогеров.

Опция 1. ИИ на основе полезности. Спроектируйте ИИ на основе полезности для некоторого NPC и реализуйте в Unreal Engine 4. Продемонстрируйте работоспособность разработанного ИИ (7 баллов).

Опция 2. Q-learning. Реализация Q-learning на Python и применение к двум окружениям. Описание задания и шаблон для решения:

<https://huggingface.co/learn/deep-rl-course/unit2/hands-on> (7 баллов)

**Спасибо
за внимание!**

itMO *re than a*
UNIVERSITY

tatiana.atyapsheva@mail.ru
abrosimov.kirill.1999@mail.ru